

# Harm.On.ica S-O-L

## « SELF-AUDIT WARNING »

« **This pseudocode is probably not yet sufficient to rebuild the entire certificate factory reliably from scratch.** It was reconstructed from one evolving implementation snapshot, so some helper calls may still hide consequential details, the fast and direct paths may drift, and neither the line-to-code correspondence nor the emitted JSON schema is mechanically enforced. A stronger future process should compare several high-quality literal pseudocode/code pairs, define a fixed rule for how much detail each operation requires, map every pseudocode line to exact source ranges, output fields, invariants, and tests, and then close the loop by asking an independent implementer to rebuild the factory from the pseudocode alone. Differences between the rebuilt artifacts and the reference certificates should drive repeated synchronized revisions of both code and pseudocode until the round trip is reproducible. »

**PROVISIONAL DESCRIPTION - NOT A STANDALONE SPECIFICATION**

**Algorithm 1.** Run one RELAY case and coordinate every subordinate method.

**Input:** Integer  $q$ , term limit  $L$ , output root  $O$ , and flags `deriveODEDirect`, `skipValidate`.

**Output:** A fresh case directory containing the fast certificate, semantic annotations, optional direct ODE certificate, independent validation, timing data, and checksums.

**Role:** Top-level orchestration only. Mathematical work is delegated to Algorithms 2–11.

---

```

1: function RELAYCASE( $q, L, O, \text{deriveODEDirect}, \text{skipValidate}$ )
2:   if  $q < 2$  then
3:     return Error: require  $q \geq 2$ 
4:    $t_0 \leftarrow \text{WALLCLOCK}()$ 
5:    $Q \leftarrow \text{RESOLVE}(O) / ("q" \parallel \text{STRING}(q))$ 
6:   if EXISTS( $Q$ ) then
7:     REMOVETREE( $Q$ )
8:    $B \leftarrow \text{BUILDFASTCASE}(q, Q, L)$ 
9:   ADDSEMANTICOBJECTS( $B$ )
10:  if deriveODEDirect then
11:     $\rho \leftarrow \text{PARSE}(B.\text{objects}.\text{input\_polynomials}.\text{rho\_q})$ 
12:     $D_{\text{dir}} \leftarrow \text{BUILDDIRECTODE}(q, \rho)$ 
13:     $P_{\text{fast}} \leftarrow \text{PARSEVECTOR}(B.\text{objects}.\text{p\_recurrence}.\text{coefficients})$ 
14:     $P_{\text{dir}} \leftarrow \text{PARSEVECTOR}(D_{\text{dir}}.\text{induced\_coefficient\_recurrence}.\text{primitive\_coefficients})$ 
15:    assert LENGTH( $P_{\text{fast}}$ ) = LENGTH( $P_{\text{dir}}$ )
16:    for  $r = 0, \dots, q - 1$  do
17:      assert EXPAND( $P_{\text{fast}, r} - P_{\text{dir}, r}$ ) = 0
18:    store  $D_{\text{dir}}$  and the zero component residuals in  $B$ 
19:  else
20:    store {requested:false,status:not_run} in  $B.\text{objects}.\text{direct\_derivative\_reduction}$ 
21:  set  $B.\text{format} \leftarrow \text{RELAY-CT-annotated-case-v0.2}$ 
22:  set artifact version, privacy-safe command/environment, total wall time, peak RSS, and direct-mode flag
23:  WRITEJSON( $Q/\text{case.json}, B, \text{sorted=true}, \text{indent=2}$ )
24:  if  $\neg \text{skipValidate}$  then
25:     $W \leftarrow \text{VALIDATECASE}(Q/\text{case.json})$ 
26:    store ( $W.\text{pass}, W.\text{passed\_checks}, W.\text{total\_checks}, W.\text{elapsed\_seconds}$ ) in  $B.\text{statistics}$ 
27:    rewrite  $\text{case.json}$ ; write  $W$  to  $\text{validation.json}$ 
28:  else
29:    PROGRESS(VALENATION, SKIPPED)
30:  for each regular file  $f$  in  $Q$  except SHA256SUMS.txt do
31:    append SHA256( $f$ ) and RELATIVEPATH( $f, Q$ ) to checksum lines
32:  write sorted checksum lines to  $Q/\text{SHA256SUMS.txt}$ 
33:  PROGRESS(COMPLETE,  $q, \text{WALLCLOCK}() - t_0, Q$ ); return  $B$ 

```

▷ the only CLI range guard  
 ▷ replace, never merge  
 ▷ Algorithm 2  
 ▷ Algorithm 8  
 ▷ Algorithm 10  
 ▷ Algorithm 11; JSON only

---

**Composition.** Algorithm 1  $\rightarrow 2 \rightarrow (3,4,5,6,7)$ , then  $8 \rightarrow 9$ ; optionally 10; finally 11. Every subordinate algorithm has its own page so no mathematical phase is hidden inside the driver.

*Implementation map.* Literal transcription of the control flow in `relay_factory_v02.py::main` and `write_checksums`.

**Algorithm 2.** Build the complete fast shift-reduction case.

**Input:**  $q \geq 2$ , empty directory  $Q$ , term limit  $L$ .

**Output:** Base JSON blob  $B$ , exact sidecars, recurrence  $P$ , certificate numerator  $N$ , and all fast-path checks.

**Role:** Expose the real phase ordering of `build_case`; Algorithms 3–6 expand its four dense blocks, and Algorithm 7 is its shared normalization helper.

---

```

1: function BUILDFASTCASE( $q, Q, L$ )
2:   start wall, CPU, peak-memory, and per-stage timers; create  $Q$ 
3:    $D \leftarrow 1 - \sum_{k=2}^q \binom{q}{k} u^{k-1}$ ;  $\rho \leftarrow uD$ 
4:   assert  $\deg_u \rho = q$  ▷ declared polynomial space dimension
5:    $(G, G^{-1}, E, U, V, J, \det G) \leftarrow \text{BUILDGUVOPERATORS}(q, \rho)$  ▷ Algorithm 3
6:    $(X_{\text{full}}, X, P, \mathcal{C}, \delta_X) \leftarrow \text{REDUCESHIFTSANDSOLVERECURRENCE}(q, \rho, U, V, J)$  ▷ Algorithm 4
7:    $(N, R, \Lambda) \leftarrow \text{ASSEMBLEFASTCERTIFICATE}(q, \rho, P, \mathcal{C})$  ▷ Algorithm 5
8:    $B \leftarrow \text{AUDITANDSERIALIZEFASTCASE}(q, Q, L, D, \rho, G, G^{-1}, E, U, V, J, X_{\text{full}}, X, P, N, R, \Lambda, \delta_X)$  ▷ Algorithm 6
9:   record wall/CPU/RSS and every completed phase duration in  $B.\text{statistics}$ 
10:  write the base version of  $B$  to  $Q/\text{case.json}$ 
11:  return  $B$ 

```

---

**Phase order:** input polynomials  $\rightarrow G$  assembly  $\rightarrow$  determinant  $\rightarrow$  inversion  $\rightarrow U, V, J \rightarrow$  pole lowering  $\rightarrow$  remainder matrix  $\rightarrow$  kernel  $\rightarrow$  rank witness  $\rightarrow$  certificate  $\rightarrow$  telescoping check  $\rightarrow$  terms  $\rightarrow$  strings  $\rightarrow$  serialization.

**Primary failure returns:**  $\deg \rho \neq q$ ; singular  $G$ ; failed  $G[U; V] = E$ ; nonzero top remainder row; wrong rank/nullity;  $XP \neq 0$ ; zero rank-witness minor; nonpolynomial certificate scale or numerator; nonzero telescoping residual; failed term/string/ $q = 4$  checks.

**Progress policy:** Emit a stage label before each expensive operation and a completion message after determinant, inversion, split, kernel, rank witness, certificate canonicalization, telescoping, terms, strings, and serialization.

*Implementation map.* This page is intentionally short: it corresponds to the single real Python function, while pages 4–7 are literal factorizations of its contiguous code blocks for readability.

**Algorithm 3.** Construct the exact  $G, U, V, J$  reduction operators.

**Input:**  $q$  and  $\rho(u)$  with  $\deg \rho = q$ .  
**Output:**  $G \in \mathbb{Z}^{2q \times 2q}$ ,  $G^{-1}$ , embedding  $E, U, V, J \in \mathbb{Q}^{q \times q}$ , and  $\det G$ .  
**Role:** Encode  $w = \rho a - \rho' b$  coefficientwise and extract the quotient/remainder-certificate maps used by every pole-lowering step.

---

```

1: function BUILDGUVOPERATORS( $q, \rho$ )
2:   create symbols  $a_0, \dots, a_{q-1}, b_0, \dots, b_{q-1}$ 
3:    $a(u) \leftarrow \sum_{j=0}^{q-1} a_j u^j$ ;  $b(u) \leftarrow \sum_{j=0}^{q-1} b_j u^j$ 
4:    $w(u) \leftarrow \text{EXPAND}(\rho a - \rho' b)$ 
5:    $y \leftarrow (a_0, \dots, a_{q-1}, b_0, \dots, b_{q-1})$ 
6:    $G \leftarrow 0_{2q \times 2q}$ 
7:   for  $i = 0, \dots, 2q - 1$  do
8:      $c_i \leftarrow [u^i]w$ 
9:     for  $j = 0, \dots, 2q - 1$  do
10:       $G_{ij} \leftarrow \partial c_i / \partial y_j$  ▷ literal coefficient map
11:    $g_G \leftarrow \text{DET}(G)$ 
12:   if  $g_G = 0$  then
13:     return Error: singular  $G$ 
14:    $G^{-1} \leftarrow \text{INVERSE}(G)$  ▷ exact arithmetic
15:    $E \leftarrow 0_{2q \times q}$ 
16:   for  $i = 0, \dots, q - 1$  do
17:      $E_{ii} \leftarrow 1$  ▷ embed degree <  $q$  numerators
18:    $U \leftarrow (G^{-1})_{0:q, :} E$ 
19:    $V \leftarrow (G^{-1})_{q:2q, :} E$ 
20:    $J \leftarrow 0_{q \times q}$ 
21:   for  $j = 1, \dots, q - 1$  do
22:      $J_{j-1, j} \leftarrow j$  ▷  $\text{poly}(Jv) = \partial_u \text{poly}(v)$ 
23:    $Z \leftarrow G \begin{bmatrix} U \\ V \end{bmatrix} - E$ 
24:   for each entry  $z$  of  $Z$  do
25:     assert  $z = 0$ 
26:   return ( $G, G^{-1}, E, U, V, J, g_G$ )

```

---

The output satisfies, for every coefficient vector  $x \in \mathbb{Q}(n)^q$ ,

$$\rho \text{poly}(Ux) - \rho' \text{poly}(Vx) = \text{poly}(x).$$

Consequently the one-step operator used later is  $T_m x = Ux - JVx/m$ . The matrix  $J$  is stored explicitly rather than inferred during replay.

*Implementation map.* Directly corresponds to the symbol creation, nested coefficient/differentiation loops, determinant, inverse, embedding, row split, derivative matrix, and exact split check in `build_case`.

**Algorithm 4.** Reduce all shifts, build  $X$ , and extract the primitive recurrence.

**Input:**  $q, \rho$ , and exact matrices  $U, V, J$ .  
**Output:**  $X_{\text{full}} \in \mathbb{Q}(n)^{q \times q}$ ,  $X \in \mathbb{Q}(n)^{(q-1) \times q}$ , primitive  $P \in \mathbb{Z}[n]^q$ , stored lowering chains  $\mathcal{C}$ , and nonzero rank witness  $\delta_X$ .  
**Role:** Apply the same literal pole-lowering update to each shift and use the unique one-dimensional kernel as the recurrence.

---

```

1: function REDUCESHIFTSANDSOLVETRECURRENCE( $q, \rho, U, V, J$ )
2:    $e_0 \leftarrow (1, 0, \dots, 0)^T$ ;  $\mathcal{X} \leftarrow []$ ;  $\mathcal{C} \leftarrow []$ 
3:   for  $r = 0, \dots, q-1$  do
4:      $x \leftarrow e_0$ ;  $C_r \leftarrow []$ 
5:     for  $\ell = r, r-1, \dots, 1$  do
6:        $m \leftarrow n + \ell - 1$ 
7:        $c \leftarrow Vx$ 
8:        $z \leftarrow Ux - Jc/m$  ▷ certificate vector before lowering
9:       for  $i = 0, \dots, q-1$  do
10:         $z_i \leftarrow \text{CANCEL}(z_i)$ 
11:        append  $\{\text{lowering\_index} : \ell, \text{pole\_parameter} : m, \text{input} : x, \text{certificate} : c, \text{output} : z\}$  to  $C_r$ 
12:         $x \leftarrow z$  ▷ one denominator power removed
13:     if  $r = 0$  then
14:        $s_r \leftarrow 1$ 
15:     else
16:        $s_r \leftarrow \text{CANCEL}(n/(n+r))$ 
17:     append  $\text{CANCELENTRYWISE}(s_r x)$  to  $\mathcal{X}$ ; append  $C_r$  to  $\mathcal{C}$ 
18:    $X_{\text{full}} \leftarrow \text{HORIZONTALJOIN}(\mathcal{X}_0, \dots, \mathcal{X}_{q-1})$ 
19:   for  $r = 0, \dots, q-1$  do
20:     assert  $\text{CANCEL}((X_{\text{full}})_{q-1, r}) = 0$ 
21:    $X \leftarrow (X_{\text{full}})_{0:q-1, :}$ 
22:   convert  $X$  to an exact dense-domain matrix over  $\mathbb{Q}(n)$ 
23:    $r_X \leftarrow \text{RANK}(X)$ ;  $K \leftarrow \text{NULLSPACE}(X)$ 
24:   assert  $r_X = q-1$  and  $\text{SHAPE}(K) = (1, q)$ 
25:    $P \leftarrow \text{PRIMITIVEVECTOR}(K_{0, :}, n)$  ▷ Algorithm 7
26:    $z_P \leftarrow XP$ 
27:   for each entry  $z$  of  $z_P$  do
28:     assert  $\text{CANCEL}(z) = 0$ 
29:    $M \leftarrow X_{:, 0:q-1}$ ;  $\delta_X \leftarrow \text{DET}_{\mathbb{Q}(n)}(M)$ 
30:   assert  $\delta_X \neq 0$  ▷ explicit rank witness
31:   return  $(X_{\text{full}}, X, P, \mathcal{C}, \delta_X)$ 

```

---

For shift  $r$ , the input integrand ratio contributes the prefactor  $s_r = n/(n+r)$  and begins at pole depth  $r$ . Each loop record is retained, because the certificate numerator and the later JSON replay both require the exact intermediate  $Vx$  vectors.

*Implementation map.* Direct transcription of the two nested lowering loops,  $X_{\text{full}}$  construction, top-row assertion, exact-domain rank/nullspace call, primitive normalization, kernel residual, and leading-minor determinant.

**Algorithm 5.** Assemble  $R = N/\rho^{q-2}$  and verify the cleared telescoping identity.

**Input:**  $q$ ,  $\rho$ , primitive recurrence  $P$ , and stored shift chains  $\mathcal{C}$ .  
**Output:** Polynomial numerator  $N(n, u)$ , rational certificate  $R(n, u)$ , and polynomial shift scales  $\Lambda = (\lambda_0, \dots, \lambda_{q-1})$ .  
**Role:** Convert every stored  $V$ -certificate vector into its common-denominator contribution, then verify the exact identity before any JSON annotation.

---

```

1: function ASSEMBLEFASTCERTIFICATE( $q, \rho, P, \mathcal{C}$ )
2:    $N \leftarrow 0$ ;  $\Lambda \leftarrow []$ 
3:   for  $r = 0, \dots, q-1$  do
4:      $\lambda_r \leftarrow \text{CANCEL}(P_r(n) n / (n+r))$ 
5:     assert  $\text{DENOMINATOR}(\lambda_r) = 1$ 
6:     append  $\lambda_r$  to  $\Lambda$  ▷ polynomial shifted recurrence scale
7:   for  $r = 1, \dots, q-1$  do
8:     for each record  $d$  in  $\mathcal{C}_r$  do
9:        $\ell \leftarrow d.\text{lowering\_index}$ ;  $m \leftarrow d.\text{pole\_parameter}$ ;  $c \leftarrow d.\text{certificate}$ 
10:       $C(u) \leftarrow \sum_{j=0}^{q-1} c_j u^j$ 
11:       $\eta \leftarrow \text{CANCEL}(\lambda_r C(u) \rho^{q-1-\ell} / m)$ 
12:       $N \leftarrow N + \eta$  ▷ common denominator  $\rho^{q-1}$ 
13:    $N \leftarrow \text{CANCEL}(N)$ 
14:   assert  $\text{DENOMINATOR}(N) = 1$ 
15:    $N \leftarrow \text{POLY}_{\mathbb{Q}[n, u]}(N).\text{AsEXPRESSION}()$ 
16:    $R \leftarrow N / \rho^{q-2}$ 
17:    $F \leftarrow \rho \partial_u N - (n+q-2)\rho' N$ 
18:   for  $r = 0, \dots, q-1$  do
19:      $F \leftarrow F - \lambda_r \rho^{q-1-r}$ 
20:    $F \leftarrow \text{POLY}_{\mathbb{Q}[n, u]}(F)$ 
21:   assert  $F = 0$  ▷ cleared exact certificate identity
22:   return  $(N, R, \Lambda)$ 

```

---

The verified polynomial identity is

$$\rho N_u - (n+q-2)\rho' N = \sum_{r=0}^{q-1} \lambda_r(n) \rho^{q-1-r}, \quad \lambda_r(n) = P_r(n) \frac{n}{n+r}.$$

After division by  $n\rho^{n+q-1}$  this becomes  $\sum_r P_r(n) H_{n+r} = \partial_u(RH_n)$ . No numerical substitution is used.

*Implementation map.* Corresponds to the partial-certificate accumulation, polynomial canonicalization, recurrence-scale construction, and exact cleared residual in `build_case`.

**Algorithm 6.** Audit finite data, write sidecars, and encode the base JSON object graph.

**Input:** All exact fast-path objects, term limit  $L$ , and output directory  $Q$ .

**Output:** Base blob  $B$ , dense/sparse text sidecars, exact finite checks, statistics, and an initial `case.json`.

**Role:** Keep combinatorial evidence, linear-algebra evidence, certificate evidence, and serialization evidence together without modifying the mathematics.

---

```

1: function AUDITANDSERIALIZEFASTCASE( $q, Q, L, D, \rho, G, G^{-1}, E, U, V, J, X_{\text{full}}, X, P, N, R, \Lambda, \delta_X$ )
2:    $A_k \leftarrow \text{PROFILECOUNT}(q, k)$  for  $k = 0, \dots, L$  ▷ exact multinomial profile sum
3:    $\mathcal{R} \leftarrow []$ 
4:   for  $k = 1, \dots, L - q + 1$  do
5:      $e_k \leftarrow \sum_{r=0}^{q-1} P_r(k) A_{k+r}$ ; append  $(k, e_k)$  to  $\mathcal{R}$ ; assert  $e_k = 0$ 
6:    $e_0 \leftarrow \sum_{r=0}^{q-1} P_r(0) A_r$  ▷ stored separately; recurrence begins at  $n = 1$ 
7:   for  $k = 1, 2, 3$  do
8:      $S_k \leftarrow \text{PLAINTEXTTREES}(q, k)$  ▷ recursive normalized  $q$ -slot grammar
9:     assert  $|S_k| = A_k$ ; store SHA256 of one-tree-per-line encoding and 80-column rendering
10:  if  $q = 4$  then
11:    set  $P_4^{\text{worked}}$  to the four fixed dissertation/worked-example polynomials
12:    for  $r = 0, 1, 2, 3$  do
13:      assert  $\text{EXPAND}(P_r - P_{4,r}^{\text{worked}}) = 0$ 
14:  write  $G, U, V, J$  as dense TSV files
15:  write nonzero entries of  $X$  as X_remainders_sparse.tsv
16:  write partial_certificates.tsv; retain indices and pole parameters, but write stored-in-generator in the three expression fields
17:  write plaintext strings, recurrence/certificate report, and term/recurrence residual table
18:  encode exact matrices with explicit shapes and string entries in B.objects.matrices
19:  encode  $D, \rho, P, N, R$ , formula labels, identities, term list, residual list, rank witness, degrees, counts, timing, CPU, RSS, and  $q = 4$  status
20:  set checks:  $G[U; V] = E$ ,  $XP = 0$ , rank/nullity, nonzero minor, polynomial scales, polynomial  $N$ , cleared certificate zero, term residuals zero, string
    counts, and optional  $q = 4$  match
21:  write sorted  $B$  to  $Q/\text{case.json}$ ; return  $B$ 

```

---

**Named helpers left outside the certificate core.** `PROFILECOUNT` recursively enumerates arity profiles and sums exact multinomial weights; `PLAINTEXTTREES` recursively distributes leaf counts among  $q$  ordered slots, retains internal layers with at least two occupied slots, and rejects duplicate strings. These helpers validate the produced recurrence; they do not participate in  $G, U, V, J$ , kernel extraction, or certificate construction.

*Implementation map.* Literal phase order and output policy from the second half of `build_case`. The placeholder TSV behavior is stated because the replayable chains are added later by Algorithm 8.

**Algorithm 7.** Normalize a one-dimensional rational-function kernel vector canonically.

**Input:** Nonzero vector  $p = (p_0, \dots, p_s)$  over  $\mathbb{Q}(t)$ .

**Output:** A sign-normalized primitive vector in  $\mathbb{Z}[t]^{s+1}$  spanning the same one-dimensional subspace.

**Role:** Make independently computed nullspace bases comparable componentwise rather than only up to a rational-function scalar.

---

```

1: function PRIMITIVEVECTOR( $p, t$ )
2:   for  $i = 0, \dots, s$  do
3:      $p_i \leftarrow \text{CANCEL}(p_i)$ 
4:      $d_i(t) \leftarrow \text{POLY}_{\mathbb{Q}[t]}(\text{DENOMINATOR}(p_i))$  for  $i = 0, \dots, s$ 
5:      $d(t) \leftarrow d_0(t)$ 
6:     for  $i = 1, \dots, s$  do
7:        $d(t) \leftarrow \text{LCM}(d(t), d_i(t))$ 
8:      $f_i(t) \leftarrow \text{POLY}_{\mathbb{Q}[t]}(\text{CANCEL}(p_i d))$  for  $i = 0, \dots, s$ 
9:      $h(t) \leftarrow f_0(t)$ 
10:    for  $i = 1, \dots, s$  do
11:       $h(t) \leftarrow \text{GCD}(h(t), f_i(t))$ 
12:    for  $i = 0, \dots, s$  do
13:       $f_i(t) \leftarrow \text{EXACTQUOTIENT}(f_i(t), h(t))$ 
14:     $c \leftarrow 1$ 
15:    for every rational coefficient  $a$  of every  $f_i$  do
16:       $c \leftarrow \text{INTEGERLCM}(c, \text{DENOMINATOR}(a))$ 
17:    for  $i = 0, \dots, s$  do
18:       $f_i(t) \leftarrow \text{POLY}_{\mathbb{Z}[t]}(c f_i(t))$ 
19:     $g \leftarrow 0$ 
20:    for  $i = 0, \dots, s$  do
21:       $g \leftarrow \text{INTEGERGCD}(g, |\text{CONTENT}(f_i)|)$ 
22:    if  $g > 1$  then
23:      for  $i = 0, \dots, s$  do
24:         $f_i(t) \leftarrow f_i(t)/g$ 
25:    if  $\text{LEADINGCOEFFICIENT}(f_s) < 0$  then
26:      for  $i = 0, \dots, s$  do
27:         $f_i(t) \leftarrow -f_i(t)$ 
28:    return  $(f_0(t), \dots, f_s(t))^T$ 

```

---

The order of operations matters. Polynomial denominator clearing precedes polynomial gcd removal; rational coefficient clearing precedes integer content removal; the final sign is determined by the leading coefficient of the last component. Algorithms 4 and 10 use this same canonical convention, and Algorithm 11 recomputes it independently.

*Implementation map.* Exact common logic appears three times in v0.2.2. This page states the common algorithm once while preserving the literal ordering used in all implementations.



**Algorithm 8.** Add self-contained mathematical objects and replayable lowering chains.

**Input:** Base blob  $B$  returned by Algorithm 2.

**Output:** The same blob augmented with sequence/OGF semantics, provenance, declared bases, exact pole-lowering chains, recurrence-derived ODE data, and verification labels.

**Role:** Make `case.json` understandable and replayable without access to Python memory or project history.

---

```

1: function ADDSEMANTICOBJECTS( $B$ )
2:    $q \leftarrow B.q$ ; parse  $D, \rho, P$  from labeled strings; parse emitted terms  $A_0, \dots, A_L$ 
3:    $h \leftarrow \lfloor \sqrt{q} \rfloor$ ; set grid shape to  $(h, h)$  if  $h^2 = q$ , otherwise null
4:   add sequence family  $a_q(n)$ ,  $A_q(x) = \sum_{n \geq 0} a_q(n)x^n$ ,  $A_q(0) = 1$ , and  $(q+1)A_q = q + x + A_q^q$ 
5:   add normalized positional  $q$ -slot tree model, symbols, retention rule, and  $T_q = x + \sum_{k=2}^q \binom{q}{k} T_q^k$ 
6:   add  $T_q = A_q - 1$ ,  $T_q = x/D_q(T_q)$ , Lagrange coefficient formula, residue formula, and  $H_{q,n} = 1/(n\rho^n)$ 
7:   add shift ratio, logarithmic  $u$ -derivative, provenance nodes/edges, coefficient order, and all declared matrix bases
8:   parse emitted  $U, V, J$ ;  $e_0 \leftarrow (1, 0, \dots, 0)^T$ ;  $\mathcal{L} \leftarrow []$ 
9:   for  $r = 0, \dots, q-1$  do
10:     $x \leftarrow e_0$ ;  $S_r \leftarrow []$ 
11:    for  $\ell = r, r-1, \dots, 1$  do
12:       $m \leftarrow n + \ell - 1$ ;  $c \leftarrow \text{CANCELENTTRYWISE}(Vx)$ 
13:       $z \leftarrow \text{CANCELENTTRYWISE}(Ux - Jc/m)$ 
14:      store  $\ell, m, x, c, z$ , operator label  $T_m = U - JV/m$ , and certificate-term label in  $S_r$ 
15:       $x \leftarrow z$ 
16:     $s_r \leftarrow 1$  if  $r = 0$ , else  $s_r \leftarrow \text{CANCEL}(n/(n+r))$ 
17:    append shift, scale, initial vector, all steps, final vector, and scaled final vector to  $\mathcal{L}$ 
18:   store  $\mathcal{L}$  as  $B.\text{objects.pole\_lowering\_chains}$ 
19:    $B.\text{objects.ode\_from\_recurrence} \leftarrow \text{RECURRENCEToODE}(P, (A_0, \dots, A_L))$ 
20:   initialize direct derivative object to  $\{\text{requested: false, status: not\_run}\}$ 
21:   independently reconstruct expected  $D$ ; store zero residual for  $D$ , zero residual for  $\rho - uD$ , integrand label,  $J$  derivative claim, fast-certificate
   pass, and recurrence-ODE checks
22:   add canonical identity strings; mutate  $B$  in place; return

```

---

▷ Algorithm 9

**Important separation.** Algorithm 8 does not discover  $P$  or  $N$ . It reparses emitted exact objects and reconstructs the lowering chains so the JSON contains the intermediate state that `partial_certificates.tsv` intentionally omits.

*Implementation map.* One-to-one phase transcription of `add_semantic_objects`; object-field prose is condensed, but every computation and replay loop is explicit.

**Algorithm 9.** Convert the  $P$ -recurrence into an inhomogeneous ODE for  $A_q(x)$ .

**Input:**  $P_0(n), \dots, P_s(n)$  and emitted terms  $A_0, \dots, A_L$ .

**Output:** Theta-form operator, ordinary derivative coefficients  $C_j(x)$ , boundary polynomial  $B_0(x)$ , coefficientwise checks, and recurrence/ODE correspondence samples.

**Role:** Translate shifts to powers of  $x$  and powers of  $\theta = xD_x$ , then expand  $\theta^k$  through Stirling numbers.

---

```

1: function RECURRENCEToODE( $P, A$ )
2:    $s \leftarrow \text{LENGTH}(P) - 1$ ;  $d \leftarrow \max_r \deg_n P_r$ ;  $C_0(x), \dots, C_d(x) \leftarrow 0$ 
3:   create auxiliary symbol  $z$ ;  $\mathcal{T} \leftarrow []$ 
4:   for  $r = 0, \dots, s$  do
5:      $Q_r(z) \leftarrow \text{EXPAND}(P_r(z - r))$ ;  $h_r \leftarrow s - r$ 
6:     append  $(r, h_r, Q_r(\theta), \text{source} = P_r(\theta - r))$  to  $\mathcal{T}$ 
7:     for each monomial  $q_{rk} z^k$  in  $Q_r$  do
8:       for  $j = 0, \dots, k$  do
9:          $C_j(x) \leftarrow C_j(x) + q_{rk} \begin{Bmatrix} k \\ j \end{Bmatrix} x^{h_r+j}$ 
10:    for  $j = 0, \dots, d$  do
11:       $C_j(x) \leftarrow \text{EXPAND}(C_j(x))$ 
12:     $B_0(x) \leftarrow 0$ 
13:    for  $r = 0, \dots, s$  do
14:      for  $m = 0, \dots, r$  do
15:         $B_0(x) \leftarrow B_0(x) + x^{s-r+m} P_r(m - r) A_m$ 
16:     $B_0 \leftarrow \text{EXPAND}(B_0)$ 
17:     $A_{\text{trunc}}(x) \leftarrow \sum_{k=0}^L A_k x^k$ 
18:     $E(x) \leftarrow \text{EXPAND}(\sum_{j=0}^d C_j(x) D_x^j A_{\text{trunc}}(x) - B_0(x))$ 
19:     $e_{\max} \leftarrow \max(-1, L - d)$ 
20:    for  $e = 0, \dots, e_{\max}$  do
21:      assert  $[x^e]E(x) = 0$ 
22:     $n_{\max} \leftarrow \min(L - s, \max(0, e_{\max} - s))$ 
23:    for  $k = 1, \dots, n_{\max}$  do
24:       $r_k \leftarrow \sum_{r=0}^s P_r(k) A_{k+r}$ ;  $o_k \leftarrow [x^{k+s}]E(x)$ 
25:      assert  $o_k - r_k = 0$ ; store  $(k, o_k, r_k, 0)$ 
26:    return theta terms  $\mathcal{T}$ , coefficients  $(C_j)$ ,  $B_0$ , safe ranges, samples, and pass records

```

---

The emitted identity is  $L_q(x, \theta)A_q(x) = B_q(x)$  with  $L_q = \sum_{r=0}^s x^{s-r} P_r(\theta - r)$ . The boundary polynomial is not optional: it accounts exactly for indices exposed when the shifted recurrence is summed from its valid starting index.

*Implementation map.* Literal loops, bounds, residual construction, safe-exponent rule, and sample correspondence from the real function.

**Algorithm 10.** Derive an ODE certificate directly by repeated parameter differentiation.

**Input:**  $q, \rho(u)$ , so  $g(x, u) = \rho(u) - x$ .

**Output:**  $G_x, U_x, V_x, J$ , derivative-lowering chains, primitive ODE coefficients  $C_j(x)$ , polynomial certificate numerator  $N_x(x, u)$ , exact identity checks, and an induced primitive recurrence.

**Role:** Provide an independent path from the integrand  $F = 1/g$  to an ODE for  $A'_q(x)$ ; agreement with the fast recurrence is tested by Algorithm 1.

---

```

1: function BUILDDIRECTODE( $q, \rho$ )
2:    $g \leftarrow \text{EXPAND}(\rho - x)$ ; start stage timer
3:   create  $a_i, b_i$ ; set  $a = \sum_{i < q} a_i u^i, b = \sum_{i < q} b_i u^i, w = ga - g_u b$ 
4:   build  $G_x \in \mathbb{Q}[x]^{2q \times 2q}$  by the same nested coefficient/partial-derivative loops as Algorithm 3
5:    $d_x \leftarrow \text{FACTOR}(\det G_x)$ ; assert  $d_x \neq 0$ ;  $G_x^{-1} \leftarrow \text{INVERSE}(G_x)$  over  $\mathbb{Q}(x)$ 
6:   construct  $E, U_x, V_x, J$  exactly as in Algorithm 3; assert  $G_x[U_x; V_x] = E$ 
7:    $\mathcal{X} \leftarrow []$ ;  $\mathcal{D} \leftarrow []$ ;  $\mathcal{I} \leftarrow []$ 
8:   for  $j = 0, \dots, q-1$  do
9:      $v \leftarrow j!e_0$ ;  $S_j \leftarrow []$ ;  $I_j \leftarrow []$ 
10:    for  $m = j, j-1, \dots, 1$  do
11:       $c \leftarrow \text{CANCELENTRYWISE}(V_x v)$ 
12:       $z \leftarrow \text{CANCELENTRYWISE}(U_x v - Jc/m)$ 
13:      append  $(m, v, c, z)$  to  $S_j$ ; append  $(m, c)$  to  $I_j$ ;  $v \leftarrow z$ 
14:    append  $v$  to  $\mathcal{X}$ ; append full derivative record  $S_j$  to  $\mathcal{D}$ ; append  $I_j$  to  $\mathcal{I}$ 
15:    $X_{x,\text{full}} \leftarrow \text{HORIZONTALJOIN}(\mathcal{X})$ ; assert its last row is zero;  $X_x \leftarrow X_{x,\text{full}}[0 : q-1, :]$ 
16:   compute exact rank/nullspace over  $\mathbb{Q}(x)$ ; assert rank  $q-1$  and nullspace shape  $(1, q)$ 
17:    $C \leftarrow \text{PRIMITIVEVECTOR}(\text{NULLSPACE}(X_x)_{0,:}, x)$ ; assert  $X_x C = 0$ 
18:    $N_x \leftarrow 0$ 
19:   for  $j = 1, \dots, q-1$  do
20:     for each  $(m, c)$  in  $\mathcal{I}_j$  do
21:        $C_c(u) \leftarrow \sum_{i=0}^{q-1} c_i u^i$ ;  $N_x \leftarrow N_x + C_j(x)C_c(u)g^{q-1-m}/m$ 
22:    $N_x \leftarrow \text{CANCEL}(N_x)$ ; assert denominator = 1; canonicalize in  $\mathbb{Q}[x, u]$ 
23:    $L \leftarrow \sum_{j=0}^{q-1} C_j(x)j!g^{q-j-1}$ 
24:    $R \leftarrow g\partial_u N_x - (q-1)g_u N_x$ ; assert  $\text{POLY}_{\mathbb{Q}[x,u]}(L - R) = 0$ 
25:   derive raw recurrence from coefficients of  $\sum_j C_j D_x^j A'_q = 0$ ; normalize it with Algorithm 7
26:   return matrices, chains,  $C, N_x$ , induced recurrence, stage timings, and exact checks
    
```

$\triangleright D_x^j F = j!/g^{j+1}$

---

*Implementation map.* The duplicate assignment of the cleared left side in the Python source is semantically idempotent and is represented once here. All substantive loops and tests are retained.

**Algorithm 11.** Replay the emitted JSON and verify every exact identity independently.

**Input:** Path  $J_{\text{path}}$  to an emitted `case.json`; no generator memory and no TSV dependency.

**Output:** A named check dictionary, passed/total counts, overall Boolean pass, and elapsed checker time.

**Role:** Detect malformed serialization, basis disagreement, altered matrices, broken chains, invalid certificates, incorrect recurrence/ODE conversion, and direct-path incompatibility.

---

```

1: function VALIDATECASE( $J_{\text{path}}$ )
2:   parse JSON; create RECORD( $\text{name}, \text{pass}, \text{observed}$ ); parse  $q, n, u, x, \theta$ , exact expressions, vectors, and matrices
3:   record format/version and required semantic-object presence
4:   rebuild  $D_{\text{expected}} = 1 - \sum_{k=2}^q \binom{q}{k} u^{k-1}$ ; record  $D = D_{\text{expected}}$  and  $\rho = uD$ 
5:   record integrand labels, coefficient order, basis lengths, matrix-declared shapes, and encoded-entry dimensions
6:   parse  $G, G^{-1}, E, U, V, J, X_{\text{full}}, X, P, N$ ; record  $\det G \neq 0, GG^{-1} = I, G[U; V] = E$ , and exact derivative matrix  $J$ 
7:    $e_0 \leftarrow (1, 0, \dots, 0)^T$ ;  $\mathcal{X} \leftarrow []$ 
8:   for each emitted fast shift chain  $r$  do
9:      $v \leftarrow e_0$ ; record emitted initial vector  $= v$ 
10:    for each emitted step with  $(\ell, m, v_{\text{in}}, c_{\text{enc}}, v_{\text{out}})$  do
11:      record  $v = v_{\text{in}}$ ;  $c \leftarrow Vv$ ; record  $c = c_{\text{enc}}$ 
12:       $z \leftarrow \text{CANCELENTRYWISE}(Uv - Jc/m)$ ; record  $z = v_{\text{out}}$ ;  $v \leftarrow z$ 
13:      record final-before-scale; apply emitted shift scale; record final-after-scale; append to  $\mathcal{X}$ 
14:    $\hat{X}_{\text{full}} \leftarrow \text{HORIZONTALJOIN}(\mathcal{X})$ ; record  $\hat{X}_{\text{full}} = X_{\text{full}}$ ;  $X = X_{\text{full}}[0 : q-1, :]$ , last row zero,  $XP = 0$ , and rank  $q-1$ 
15:   recompute  $\lambda_r = P_r n / (n+r)$  and cleared fast residual from  $\rho, N$ ; record polynomial scales and residual zero
16:   recompute emitted finite recurrence residuals from terms; record all zero
17:   recompute Algorithm 9 from  $P$  and terms; record derivative coefficients, boundary polynomial, series residual, and coefficient correspondence equal emitted
   values
18:   inspect direct object
19:   if direct status is complete then
20:     parse  $G_x, G_x^{-1}, E_x, U_x, V_x, J_x, X_{x,\text{full}}, X_x, C, N_x$ 
21:     record determinant, inverse, split, exact  $J_x$ , and declared shapes
22:     replay every derivative chain from  $j!e_0$ ; rebuild  $X_{x,\text{full}}$ ; record final vectors, prefix,  $X_x C = 0$ , rank  $q-1$ , and leading coefficient relation
23:     recompute direct cleared certificate; normalize induced recurrence with Algorithm 7; record recurrence  $= P$ 
24:     build truncated  $A'_q(x)$  from terms; record direct ODE series residual zero on its safe range
25:   else
26:     record status = not_run and requested = false
27:    $p \leftarrow$  number of passing records;  $t \leftarrow$  total records; return format,  $q$ , source,  $(p=t)$ ,  $p$ ,  $t$ , elapsed time, and all records

```

---

**Independence boundary.** The checker trusts only the JSON syntax and declared exact strings. It reconstructs matrices, chains, kernel residuals, certificates, recurrence-to-ODE data, and optional direct data with its own parsers and its own primitive-vector implementation.

*Implementation map.* Ordered to match `validate_case`; repetitive named records are grouped by mathematical object, but no category of implemented check is omitted.